

Elaborazione di Segnali Multimediali

Segmentazione semantica

D.Cozzolino, L.Verdoliva

L'obiettivo della segmentazione semantica è associare ogni pixel dell'immagine a una classe ben definita, fornendo una mappa di etichette. Può essere vista come un'estensione della classificazione: invece di classificare l'intera immagine, vengono classificati i singoli pixel. In questa esercitazione, useremo il dataset Oxford-IIIT Pet Dataset (<https://www.robots.ox.ac.uk/~vgg/data/pets/>), dove le immagini sono foto di cani e gatti. Per ogni immagine è disponibile una mappa di segmentazione che permette di separare il cane o il gatto (foreground) dallo sfondo (background). Per definire l'architettura, la data augmentation e la funzione di loss per la segmentazione semantica useremo le librerie `segmentation-models-pytorch` e `albumentations`, installabili in Colab con la seguente istruzione:

```
!pip install -q segmentation-models-pytorch albumentations
```

Prima di iniziare, resettiamo la console e importiamo le librerie che useremo:

```
%reset -f
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.utils.data as data
import tqdm
import segmentation_models_pytorch as smp
import albumentations as A
```

1 Definizione della rete neurale

Come architettura useremo la U-Net, che è formata da due parti: l'*encoder*, detto anche *contracting path*, e il *decoder*, chiamato *expansive path*. L'encoder estrae rappresentazioni compatte di alto livello, mentre il decoder ripristina le dimensioni spaziali, producendo una mappa di segmentazione della stessa dimensione dell'immagine di ingresso. Le *skip connection* tra encoder e decoder aiutano a recuperare dettagli spaziali e migliorano la localizzazione dei bordi. Definiamo la rete U-Net specificando l'encoder, il numero di canali in ingresso e il numero di canali in uscita:

```
device = "cuda:0" if torch.cuda.is_available() else "cpu" # usa la GPU se e' disponibile
encoder_name="resnet18"
encoder_weights="imagenet"

model = smp.Unet(
    encoder_name=encoder_name,          # encoder architecture
    encoder_weights=encoder_weights,   # encoder pretraining
    in_channels=3,                      # RGB
    classes=1,                          # binary segmentation
).to(device)
```

Nel codice precedente abbiamo specificato un numero di classi pari a 1; pertanto, per ogni pixel, la rete fornirà un solo valore di logit. Per valori di logit positivi, il pixel viene predetto come foreground, mentre per valori negativi viene predetto come background. Notate inoltre che inizializziamo un encoder già pre-addestrato su ImageNet. Pertanto, per evitare overfitting, possiamo bloccare l'addestramento dei parametri dell'encoder nel seguente modo:

```
# Unfreeze everything
for p in model.parameters():
    p.requires_grad = True # attiva calcolo del gradiente

# Freeze the encoder
for p in model.encoder.parameters():
    p.requires_grad = False # deattiva calcolo del gradiente

trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
total_params = sum(p.numel() for p in model.parameters())
print(f'Parametri addestrabili: {trainable_params:,} / {total_params:,}')
```

Inoltre, per essere coerenti con il pre-addestramento utilizziamo la stessa normalizzazione. Il codice seguente serve a ottenere i parametri della normalizzazione.

```
from segmentation_models_pytorch.encoders import get_preprocessing_fn
normalize_param = get_preprocessing_fn(encoder_name, pretrained=encoder_weights)
img_mean = normalize_param.keywords['mean']
img_std = normalize_param.keywords['std']
```

2 Preparazione dei dati

Prima di definire gli oggetti dataset, occorre definire le operazioni di preprocessing, includendo anche la data augmentation per il training. A tal fine, non utilizzeremo la libreria `torchvision.transforms`, ma la libreria `albumentations`, che ha il vantaggio di applicare le operazioni di preprocessing sia all'immagine sia alla mappa di segmentazione, in modo da preservarne la coerenza.

```
transform_train = A.Compose([
    A.Resize(256, 256),
    A.Affine(scale=(0.8, 1.5), rotate=(-10, 10), p=0.5),
    A.RandomBrightnessContrast(brightness_range=(-0.3, -0.1), contrast_range=(-0.2, 0.2), p=0.5),
    A.CenterCrop(224, 224),
    A.Normalize(img_mean, img_std),
    A.ToTensorV2(),
], additional_targets={"trimap": "mask"})

transform_eval = A.Compose([
    A.Resize(256, 256),
    A.CenterCrop(224, 224),
    A.Normalize(img_mean, img_std),
    A.ToTensorV2(),
], additional_targets={"trimap": "mask"})
```

Adesso potete eseguire le seguenti istruzioni per creare gli oggetti dataset per i tre set:

```
from segmentation_models_pytorch.datasets import OxfordPetDataset
OxfordPetDataset.download("./data")
train_set = OxfordPetDataset("./data", "train", transform=transform_train)
val_set = OxfordPetDataset("./data", "valid", transform=transform_eval)
```

```
test_set = OxfordPetDataset("./data", "test", transform=transform_eval)

print(f"Train size: {len(train_set)}")
print(f"Valid size: {len(val_set)}")
print(f"Test size: {len(test_set)}")
```

Creiamo i DataLoader per ciascun set specificando il batch_size:

```
batch_size=64
train_loader = data.DataLoader(train_set, batch_size=batch_size, shuffle=True)
val_loader = data.DataLoader(val_set, batch_size=batch_size, shuffle=False)
test_loader = data.DataLoader(test_set, batch_size=batch_size, shuffle=False)
```

Visualizziamo alcuni esempi del test set. La maschera è mostrata in scala di grigi: nero per background e bianco per foreground.

```
plt.figure()
for index in range(5):
    sample = test_set[index]
    img = sample["image"].cpu().numpy().transpose(1, 2, 0)
    img = img * img_std + img_mean # Denormalizzazione
    msk = sample["mask"].cpu().numpy()

    plt.subplot(2, 5, index+1)
    plt.imshow(img)
    plt.subplot(2, 5, index+6)
    plt.imshow(msk, cmap='gray', clim=[0,1])
plt.show()
```

3 Addestramento

In questa esercitazione usiamo l'ottimizzatore NAdam e la Focal Loss. La Focal Loss è una variante della cross-entropy che assegna maggiore peso ai pixel difficili da classificare. Nella segmentazione binaria, la rete produce un tensore di dimensione $N \times 1 \times H \times W$, mentre la maschera target ha dimensione $N \times H \times W$ e contiene etichette 0 per il background o 1 per il foreground.

```
loss_function = smp.losses.FocalLoss(mode="binary")
optimizer = torch.optim.NAdam([p for p in model.parameters() if p.requires_grad], lr=1e-4)
num_epochs = 5
```

Per comodità definiamo una funzione per l'addestramento di un'epoca:

```
def train_one_epoch(model, loader, loss_function, optimizer, device):
    model.train() # abilita comportamenti specifici del training (es. dropout)
    cum_loss = 0.0
    num_images = 0
    for batch in tqdm.tqdm(loader):
        images = batch["image"].to(device)
        target = batch["mask"].to(device)

        optimizer.zero_grad() # (1) azzera i gradienti
        output = model(images) # (2) applica la rete
        loss = loss_function(output, target) # (3) calcolo del valore di loss
        loss.backward() # (4) calcolo dei gradienti
        optimizer.step() # (5) aggiornamento dei pesi
```

```

# Aggiornamento delle statistiche
cum_loss += loss.item() * images.shape[0]
num_images += images.shape[0]
train_loss = cum_loss / num_images
return train_loss

```

Definiamo un'altra funzione per la valutazione, in cui utilizziamo come indice prestazionale la Intersection over Union (IoU). Non usiamo l'accuratezza perché, nella segmentazione, può essere poco informativa quando la regione da localizzare occupa una porzione molto piccola dell'immagine.

```

def evaluate(model, loader, loss_function, device):
    model.eval() # disabilita comportamenti specifici del training
    epoch_cum_loss = 0.0
    epoch_cum_iou = 0.0
    epoch_num_images = 0
    with torch.no_grad(): # disabilita il calcolo dei gradienti
        for batch in tqdm.tqdm(loader):
            images = batch["image"].to(device)
            target = batch["mask"].to(device)

            output = model(images)
            loss = loss_function(output, target)
            predicted = output > 0
            num_img = images.shape[0]
            epoch_cum_loss += loss.item() * num_img
            tp, fp, fn, tn = smp.metrics.get_stats(
                torch.reshape(predicted, (num_img, 1, -1)).long(),
                torch.reshape(target, (num_img, 1, -1)).long(),
                mode="binary")
            epoch_cum_iou += torch.sum(smp.metrics.iou_score(tp, fp, fn, tn))
            epoch_num_images += num_img
    epoch_loss = epoch_cum_loss / epoch_num_images
    epoch_iou = epoch_cum_iou / epoch_num_images
    return epoch_loss, epoch_iou

```

Eseguiamo il training per le diverse epoche:

```

for epoch in range(num_epochs):
    # ---- Fase di TRAINING ----
    train_loss = train_one_epoch(model, train_loader, loss_function, optimizer, device)
    print(f'Epoca [{epoch}/{num_epochs}] Loss train: {train_loss:.4f}')
    # ---- Fase di VALIDAZIONE ----
    val_loss, val_iou = evaluate(model, val_loader, loss_function, device)
    print(f'Epoca [{epoch}/{num_epochs}] Loss val: {val_loss:.4f} IoU val: {val_iou:.4f}')

```

Per rendere l'esercitazione rapida impostiamo solo 5 epoche, ma in un esperimento reale sarebbe opportuno aumentare questo numero.

4 Analisi delle prestazioni

Valutiamo le prestazioni del modello sul test set:

```

test_loss, test_iou = evaluate(model, test_loader, loss_function, device)
print(f'Loss test: {test_loss:.4f} IoU test: {test_iou:.4f}')

```

Visualizziamo ora il risultato su un'immagine del test set.

```
#Test su una singola immagine
ID = 150 # indice dell'immagine nel test set

sample = test_set[ID]
x = sample["image"].unsqueeze(0) # aggiungiamo la dimensione batch

model.eval()
with torch.no_grad():
    output = model(x.to(device))
    pred = output>0 # predizione
    pred = pred[0,0].cpu().numpy() # conversione da torch a numpy

img = sample["image"].cpu().numpy().transpose(1, 2, 0) # conversione da torch a numpy
img = img * img_std + img_mean # Denormalizzazione
msk = sample["mask"].cpu().numpy()

plt.figure()
plt.subplot(1, 3, 1)
plt.imshow(img)
plt.title('immagine')
plt.subplot(1, 3, 2)
plt.imshow(msk, cmap='gray', clim=[0,1])
plt.title('ground truth')
plt.subplot(1, 3, 3)
plt.imshow(pred, cmap='gray', clim=[0,1])
plt.title('predizione')
plt.show()
```

5 Esercizi proposti

1. *Architettura*. Provate SegFormer, una rete basata su Transformer per la segmentazione; potete definirla nel seguente modo:

```
encoder_name="mit_b0"
encoder_weights="imagenet"
model = smp.Segformer(
    encoder_name=encoder_name,          # encoder architecture
    encoder_weights=encoder_weights,    # encoder pretraining
    in_channels=3,                      # RGB
    classes=1,                          # binary segmentation
).to(device)
```

2. *Funzione di loss image-wise*. La Focal Loss è una funzione valutata pixel-wise, che considera ogni pixel in maniera indipendente. Per la segmentazione semantica, è più importante che la regione predetta sia sovrapposta alla regione reale che la decisione sul singolo pixel. Per tale motivo, in letteratura sono state proposte loss image-wise, come la Dice Loss.

```
loss_function = smp.losses.DiceLoss(mode="binary")
```